

MOBILE APP TESTING: FROM A TO Z



Types of Mobile App Testing

FUNCTIONAL TESTING

Every function of an application is working as intended and acting by predetermined requirements.

USER INTERFACE (UI)

It's about testing the graphical elements to verify that they are functioning according to requirements.

USABILITY TESTING (UX)

Allows us to measure evaluation of the compliance of the application design with its functionality specified by the customer.

COMPATIBILITY TESTING

Ensures the consistent functioning of an app across devices and software ecosystems.

ACCESSIBILITY TESTING

We run mobile accessibility testing to verify that an app has been adjusted for accessibility needs.

PERFORMANCE TESTING

We can learn about an app's stability, resource consumption, and ability to withstand high loads.

LOCALIZATION TESTING

Ensures that an application has been adjusted to a target market in terms of language, cultural characteristics, and specifics of a particular country or region.

INTERNATIONALIZATION TESTING

Verifies that an app has been adjusted to a particular region.

SECURITY TESTING

Allows uncovering existing and potential vulnerabilities, threats, risks in a mobile application, and loopholes in the processes.

CHANGE-RELATED TESTING

Once developers fix bugs, QA engineers need to verify the changes and make sure the rest of the functionality is still working.

API TESTING

Helps to determine software strengths at the early stages of development. Therefore, API tests detect small errors that might become more serious while running GUI.

END-TO-END (E2E) TESTING

A QA engineer checks a user flow from the beginning to the end. It gives a clear perspective of a user journey through some part of a mobile app.

ACCEPTANCE TESTING

It is a final verification of the business functionality. The QA team goes through a product's functionality to see if the software system meets specification requirements.

Mobile App Testing Approaches

The approaches to mobile app testing are often based on *timing*.

PROACTIVE APPROACH

A test design process starts early and aims to prevent issues before even code is ready. The QA team starts with review of requirements to prevent bugs and shape software quality from the very beginning.

REACTIVE APPROACH

Implies working with an end-product, when its design and functionality are supposedly ready for use. Utilizing the reactive approach, a testing team waits to run tests at the late stage of an SDLC.



Areas to Focus on in Mobile Testing App Testing

- Installation / de-installation / update process.
- Location & VPN.
- Data synchronization.
- Interruptions of the application by calls, messages, etc.
- An ability to run the app in the background.
- Payment gateway.
- Communication (chat, audio, video).
- Touchscreen (tap, swipe, pull, shake, pinch).
- Voice commands.



Native Apps vs Hybrid Apps

NATIVE MOBILE APPS

Developed for a specific platform – Android or iOS. So if a service is present in both stores, these are two separate applications.

HYBRID MOBILE APPS

Universal, so one app can be installed on both platforms. It is possible thanks to mixing native and web elements in the development.

The testing approach will be the same for both types of apps, but...

1. The **functional** checklists or test cases will focus on the common areas.
2. Performance issues are the legacy of hybrid apps.
3. Hybrid applications require spending more time on **UI testing**
4. Testing of hybrid apps tends to require more real devices for **compatibility testing** than native app testing.

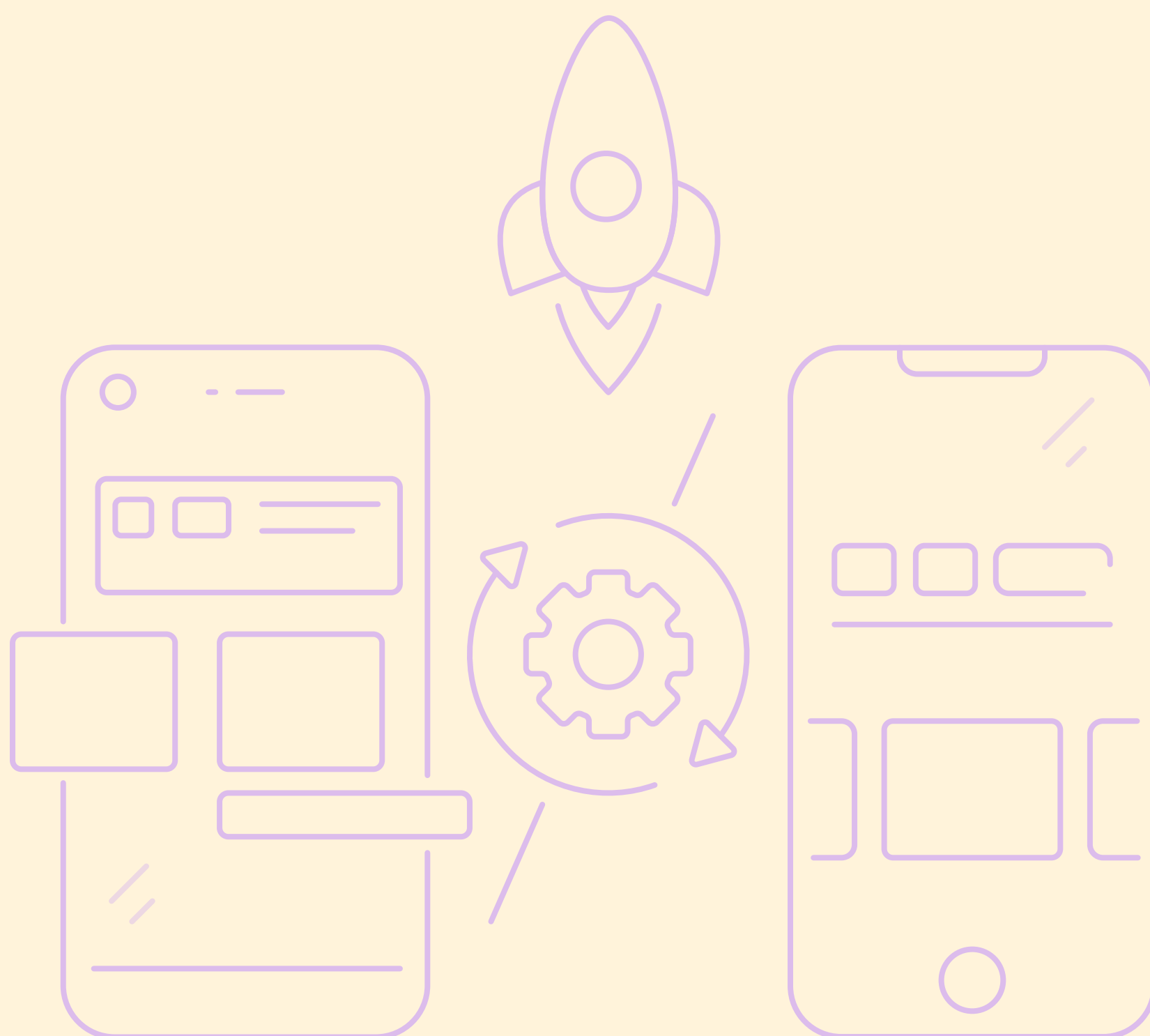


Android App Testing vs iOS App Testing

Android is represented by a vast variety of devices and OS versions. Their combinations are more numerous than in iOS. Thus, there is a bigger scope of work for Android device testing when it comes to **cross-platform** checks.

To be more specific:

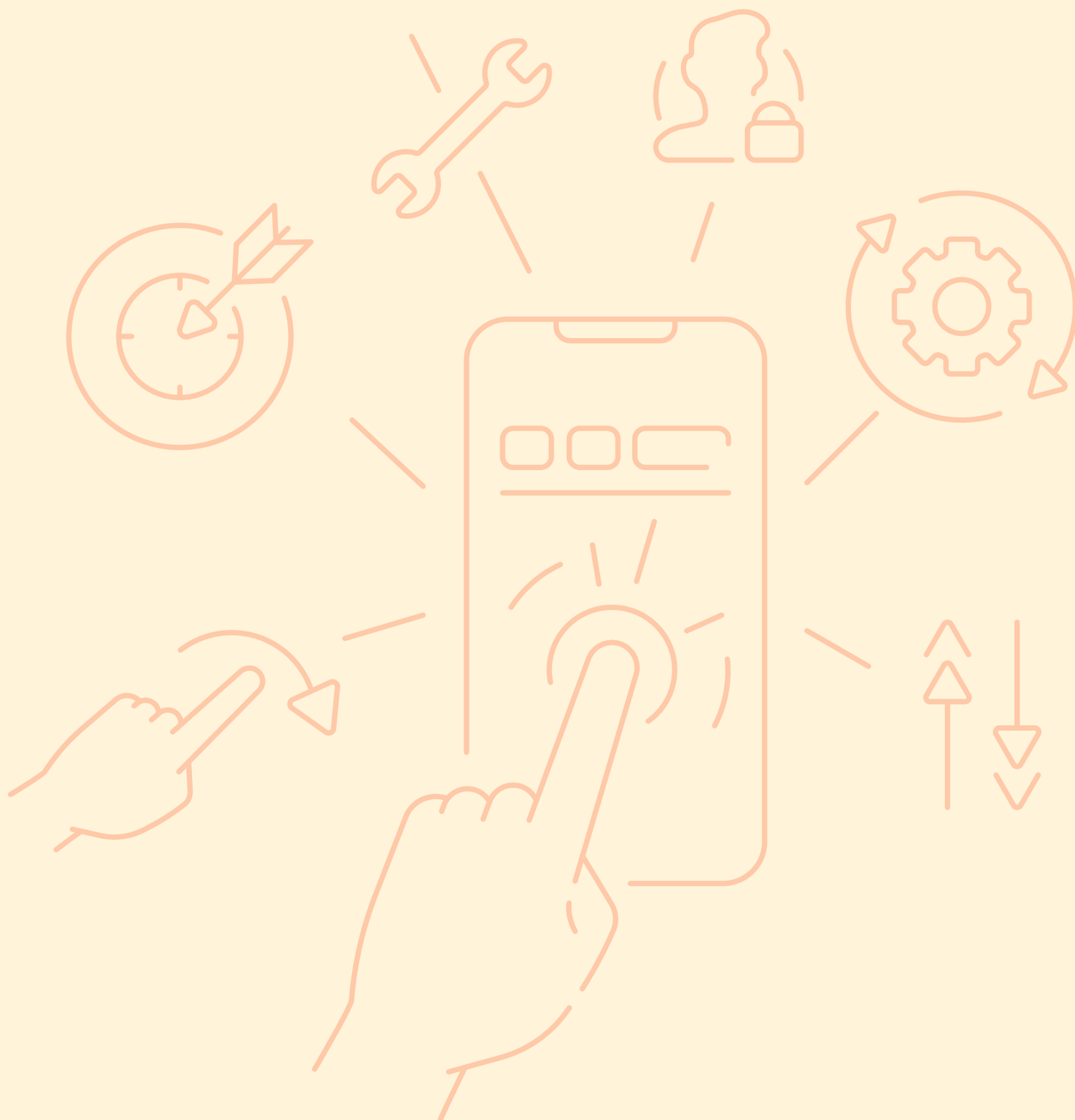
- Since Android devices come with more screen sizes and resolutions, there is more room for layout issues and interface bugs. Thus, the scope of tasks for **UI testing** is also bigger compared to iOS.
- Android offers lots of software and hardware configurations, so again, **performance and compatibility testing** would require more time.



Real-Device Testing

Testing on real devices allows a QA team to assure that an application works smoothly on a variety of smartphones. For example, only using a real device, you can properly check:

- Colors on screens with different resolutions;
- Touchscreen with all the swipes, scrolls, and taps;
- Location, camera, audio, and other device-specific features;
- Different software and hardware configurations;
- Different software and hardware configurations;





Mobile Application Testing Checklist

Mobile Application Testing Checklist

1. DEVICE SPECIFIC CHECKS

- 1.1 Can the app be installed on the device?
- 1.2 Does the app behave as designed/desired if there is an incoming call?
- 1.3 Does the app behave as designed/desired if there is an incoming SMS?
- 1.4 Does the app behave as designed/desired if the charger is connected?
- 1.5 Does the app behave as designed/desired if the charger is disconnected?
- 1.6 Does the app behave as designed/desired if the device goes to sleeping mode
- 1.7 Does the app behave as designed/desired if the device resumes from sleeping mode
- 1.8 Does the app behave as designed/desired if the device resumes from lock screen?
- 1.9 Does the app behave as designed/desired if the device is tilted?
- 1.10 Does the app behave as designed/desired if the device is shaken?
- 1.11 Does the app behave as designed/desired if a local message is coming from another app (think of: calendar reminders, to-do task etc.).
- 1.12 Does the app behave as designed/desired if a push message is coming from another app (think of: twitter mentions, whatsapp message, wordfeud invitation, etc).
- 1.13 Does the app interact with the GPS sensor correctly (switch on/off, retrieve GPS data)?
- 1.14 Is the functionality of all the buttons or keys on the device defined for this app?
- 1.15 Verify that buttons or keys which have no defined function have no unexpected behaviour on the app when activating.
- 1.16 In case there's a true "back" button available on the device does the "back" button take the user to the previous screen?
- 1.17 In case there's a true "menu" button available on the device, does the menu button show the app's menu?
- 1.18 In case there's a true "home" button available on the device, does the home button get the user back to the home screen of the device?

- 1.19 In case there's a true "search" button available on the device, does this get the user to some form of search within the app?
- 1.20 Does the app behave as designed/desired if the "Battery low" message is pushed
- 1.21 Does the app behave as designed/desired if the sound on the device is turned off?
- 1.22 Does the app behave as designed/desired if the device is in airplane mode?
- 1.23 Can the app be de-installed from the device?
- 1.24 Does the application function as expected after re-installation?
- 1.25 Can the app be found in the app store? (Check after go-live)
- 1.26 Can the app switch to different apps on the device through multitasking as designed/desired?
- 1.27 Are all touch screen positions (buttons) working when a screen protector is used.

2. NETWORK SPECIFIC CHECKS

- 2.1 Does the app behave according to specification if connected to the internet through Wi-Fi?
- 2.2 Does the app behave according to specification if connected to the internet through 3G?
- 2.3 Does the app behave according to specification if connected to the internet through 2G?
- 2.4 Does the app behave according to specification of the app is out of network reach?
- 2.5 Does the app resume working when it gets back into network reach from outside reach of the network?
- 2.6 Update transactions are processed correctly after re-establishing connection.
- 2.7 Does the app still work correctly when tethering or otherwise connected to another device
- 2.8 What happens if the app switches between networks (Wi-Fi, 3G, 2G)

3. APP SPECIFIC CHECKS

- 3.1 Has the app been tested on different type of devices and different versions of OS?
- 3.2 Stability check: if the app has a list (for instance of pictures) in it, try scrolling through it at high speed.
- 3.3 Stability check: if the app has a list (for instance of pictures) in it, try scrolling to before the first picture or behind the last picture.

- 3.4 Is downloading of the app prevented in case it's bigger than the OS allows downloading when connected to cellular networks.
- 3.5 Integration: does the app connect correctly to the different social networks (LinkedIn, twitter, facebook, etc).
- 3.6 The app does not interfere with other apps when in background/multitasking mode (using GPS, playing music, etc.).
- 3.7 Can the user print from the app (if applicable)
- 3.8 The search option in the app displays relevant results
- 3.9 Verify most common gestures used to control the app.
- 3.10 What happens if you select different options at the same time (undesired multitouch, for example – select two contacts from the phone book at the same time).
- 3.11 App name should be self explanatory
- 3.12 Does the app limit or clean the amount of cached data.
- 3.13 Reloading of data from remote service has been properly designed to prevent performance issues at server-side. (manual reloading of data can reduce the amount of server calls)
- 3.14 Does the app go to sleep mode when running in the background (prevent battery drain)

4. APP USER INTERFACE CHECKS

This checklist is based on the recommendations of Apple and some other experts. This checklist is not a substitute for a usability test, to get a good feeling of the user experience of an app a user experience test is always the most reliable method. User interface checks toelichten.

- 4.1 To keep controls as unobtrusive as possible for instance by fading them out if they are not used for a while.
- 4.2 Make it possible for users to go back to a previous screen for instance by adding a back or cancel button
- 4.3 The main function of the app should be apparent immediately. It should speak for itself.
- 4.4 Use at most one action on the screen that is highlighted as the most likely for the user. (Example: in iOS a blue button represents the default or most likely action).

- 4.5 Minimize user actions by using a picker or a table view where users can select a certain choice over a data entry field where users have to type a choice
- 4.6 In an app, the user should not be able to store files locally, outside the app sandbox.
- 4.7 In an app, the user should not be exposed to the permissions of a specific file
- 4.8 If there is a long list of data to scroll through, provide a search option above the list.
- 4.9 If performance is slow, indicate a progress status icon (“Loading...”), preferably with specific message.
- 4.10 In case of ‘live’ filtering of data while the user enters his search query, verify the performance.
- 4.11 The appearance of buttons that perform standard actions are not altered in the app (for instance: refresh, organize, trash, Reply, back, etc.)
- 4.12 Do not use standard buttons for other functions than that they are normally used for
- 4.13 The app should respond to all changes in device orientation, as per the design
- 4.14 Do not redefine gestures in your app that have a standard meaning (example: swiping from top to bottom enables the notification center)
- 4.15 Requirement to login is delayed in the app as long as possible
- 4.16 If the app is stopped at an unexpected time, user data should be saved locally and available at start-up.
- 4.17 Users should be warned of the consequences of deleting a document
- 4.18 Keyboard adjusts to expected input (for instance numbers/letters when expected).
- 4.19 Are inactive buttons clearly distinguished from active buttons?

